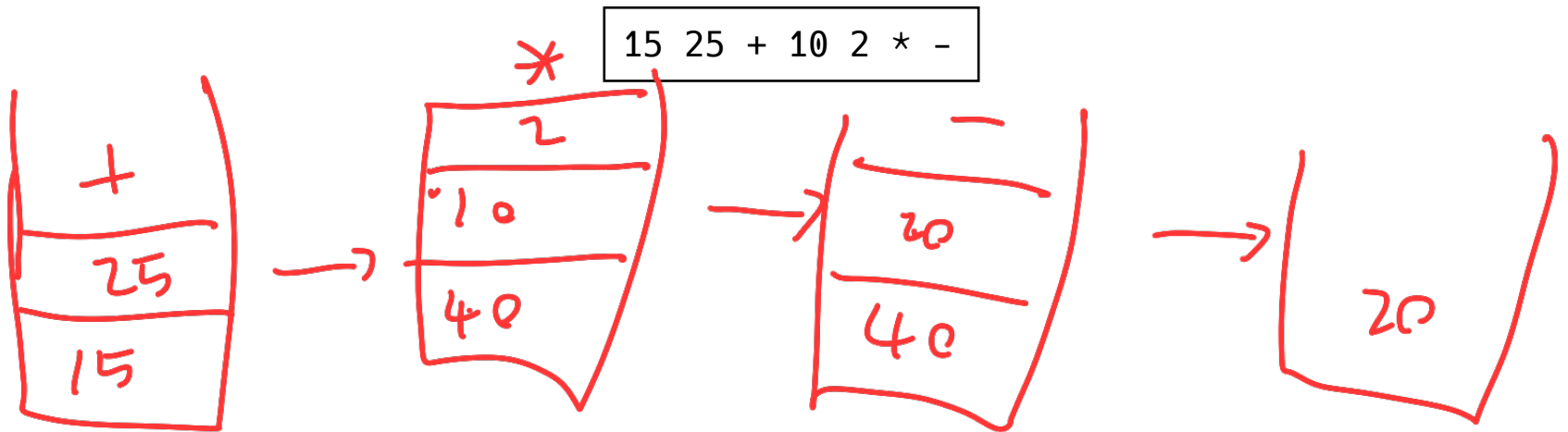


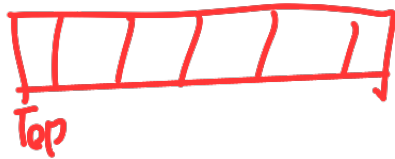
자료구조 및 알고리즘

Problem 01

스택을 이용하여 다음과 같은 후위 표현법의 식을 계산하는 과정에서 스택의 상태 변화를 설명해보시오. (단, 정수는 한번에 읽어들이한다고 가정한다.)



바나나리스트



Problem 02

본문에서 파이썬 내장 리스트를 사용하는 스택을 구현할 때 리스트의 맨 뒤를 스택의 탑으로 간주 했다. 반대로 리스트의 맨 앞을 스택의 탑으로 간주하여 다음 클래스 ListStack 코드를 바꾸어보시오.

```
class ListStack:
    def __init__(self):
        self.__stack = []

    def push(self, x):
        self.__stack.append(x)

    def pop(self):
        return self.__stack.pop()

    def top(self):
        if self.isEmpty():
            print("No element in stack")
        else:
            return self.__stack[-1]

    def isEmpty(self) -> bool:
        return not bool(self.__stack)

    def popAll(self):
        self.__stack.clear()

    def printStack(self):
        print("Stack:")
        for i in range(len(self.__stack)):
            print('stack[', i, ']:', self.__stack[i])
```

insert(0, x)

pop(0)

0

range(self.__stack, -1, -1)

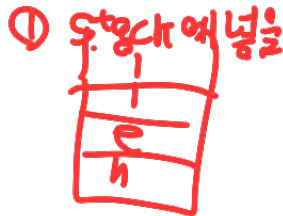
Problem 03

입력으로 들어온 문자열이 다음 집합의 원소인지 체크하는 코드를 스택을 이용하여 작성하시오.

$\{w\$w^R \mid w: \text{문자열}, w^R: w \text{의 순서를 뒤집어 놓은 것}\}$

hello\$olleh ?

① $\text{len}(str) == \text{홀수}$.



② \$이전
남은 문자열과
Stack pop 값비교.

```
def checkstr(str):
```

```
    Stack = List Stack()
```

```
    stop = 0;
```

```
    for i in range(len(str)):
```

```
        if str[i] == '$'
```

```
            stop = i
```

```
            break
```

```
        stack.push(str[i])
```

```
    for j in range(stop+1, len(str))
```

```
        if stack.isEmpty()
```

```
            return False.
```

```
        if (str[j] != stack.pop())
```

```
            return False.
```

```
        if (stack.isEmpty())
```

```
            return True.
```

```
    else
```

```
        return False.
```

복사.

Problem 04

LinkedList 타입의 객체 a의 내용을 그대로 또 다른 LinkedList 타입의 객체 b로 복사하는 코드를 작성하시오. (a의 레퍼런스 값을 b로 복사하여 두 변수가 같은 레퍼런스를 가져 내용이 같아지도록 하라는 것이 아니라 내용 복사를 요구하는 것이다.)



```
def copyStack(a, b):
    tmp = LinkedList()
    while(not a.isEmpty()):
        tmp.push(a.pop())
    while(not tmp.isEmpty()):
        item = tmp.pop()
```

Problem 05

(()) 대칭

문자열을 받아 괄호 '(', ')'의 좌우 균형이 맞는지 체크해서 균형이 맞으면 True, 맞지 않으면 False를 리턴하는 파이썬 함수 parenBalance()를 스택을 사용해서 작성하시오.

(())

풀이법

① 개수 세는 방식

② '('만 스택에 넣고 ')' 볼 때 마다.

'c' pop하는 방식

```
def parenBalance(s:string) -> bool:
```

```
    num1 = 0
```

```
    num2 = 0
```

```
    for str in range(s):
```

```
        if (str == '('):
```

```
            num1++
```

```
        if (str == ')'):
```

```
            num2++
```

```
    if (num1 == num2):
```

```
        return True:
```

```
    else  
        return false.
```

```
def ~~~~~ (s:string) -> bool:
```

```
    stack = linked list()
```

```
    for ch in range(s):
```

```
        if (str == '('):
```

```
            stack.push('(')
```

```
        if (str == ')'):
```

```
            if (stack.isEmpty())
```

```
                return false
```

```
            else stack.pop()
```

```
    if (stack.isEmpty()) : return True.  
    else : return False
```

사실 문제
예상

Problem 06

5번 문제는 단순히 한 종류의 괄호만 대상으로 하였다. 여러 종류의 괄호 균형이 다 맞는지 확인하려고 한다면 로직이 더 복잡해지는가?

```
def ParenBalance(s):
```

```
    stack = ListStack()
```

```
    for i in range(len(s)):
```

```
        if s[i] in ['(', '{', '[']:
```

```
            stack.push(s[i])
```

```
        elif s[i] in [')', '}', ']']:
```

```
            if stack.isEmpty(): return false.
```

```
            top = stack.pop()
```

```
            if (s[i] == ')' and top != '(')
```

```
                return false
```



```
return stack.isEmpty().
```